

Lab 6 - Segment Trees, Sparse Tables

February 2026

Content Summary

- **Bucket Decomposition**
- **Segment Trees**
- **Sparse Tables**
- **Practice Problems**

1 Bucket Decomposition

The problem we will focus on in this lab involves a *dynamic* array (meaning its values can change) and range queries over this array. This is a very general setting, but for simplicity we will focus on a specific version: point updates and range minimum queries. The techniques we will see can be adapted to support many other operations as well.

More precisely, we will study data structures that solve the following problem: given an array a of size n , perform q operations of the following types:

- **set** $pos\ x$ — set $a_{pos} = x$.
- **min** $l\ r$ — return $\min(a[l \dots r])$.

Let us start with a naive approach:

- For a **set** operation, we simply modify the array a . This takes $O(1)$ time.
- For a **min** query, we scan the range $[l, r]$ and compute the minimum. This takes $O(n)$ time in the worst case.

How can we optimize the **min** query? Ideally, we would like to reuse pre-computed information. Suppose we already know the minimum value on some interval $[p, q]$. When answering a query $[l, r]$ with $l \leq p \leq q \leq r$, instead of scanning all elements in $[p, q]$, we can directly use the precomputed minimum and skip to $q + 1$.

This suggests splitting the array a into several *buckets*, and storing the minimum of each bucket in a separate array.

$$a = \begin{array}{|c|c|c|c|c|c|c|c|c|c|c|c|c|c|c|} \hline a_0 & a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 & a_8 & a_9 & a_{10} & a_{11} & a_{12} & a_{13} & a_{14} \\ \hline \end{array}$$

$$b = \begin{array}{|c|c|c|c|} \hline \min(a[0\dots 3]) & \min(a[4\dots 7]) & \min(a[8\dots 11]) & \min(a[12\dots 14]) \\ \hline \end{array}$$

To answer a query $[l, r]$, we use b for buckets that are completely contained in $[l, r]$, and we scan elements from a only for the two partial buckets at the ends (if any).

For example, for a `min` query on the interval $[2, 12]$, we would only look at a_2, a_3, b_1, b_2 , and a_{12} .

A simple way to split a into buckets is to choose a bucket size d , and then create $\lceil n/d \rceil$ buckets. With this approach, we obtain the following time complexities:

- **set** — $O(d)$ in the worst case, since updating a_{pos} may require recomputing the minimum of the bucket containing pos .
- **min** — $O(\frac{n}{d} + d)$, since we do $O(d)$ work for the two ends and scan at most $O(n/d)$ whole buckets in the middle.

Note that, in some cases, **set** can be done in $O(1)$ time (if we query for sum, xor, etc).

2 Segment Trees

A segment tree is a binary tree built on top of the array. Each node represents an interval of the array and stores the minimum value on that interval.

- The root represents the interval $[0, n - 1]$.
- A node representing $[l, r]$ is split into:

$$[l, mid] \quad \text{and} \quad [mid + 1, r],$$

$$\text{where } mid = \lfloor (l + r)/2 \rfloor.$$

The leaves correspond to intervals of length 1, that is, single elements of the array.

Just like heaps, segment trees are usually stored implicitly in an array.

If we store the tree in an array t :

- The root is stored at index 1.
- For a node at index i :
 - left child is at $2i$,
 - right child is at $2i + 1$.

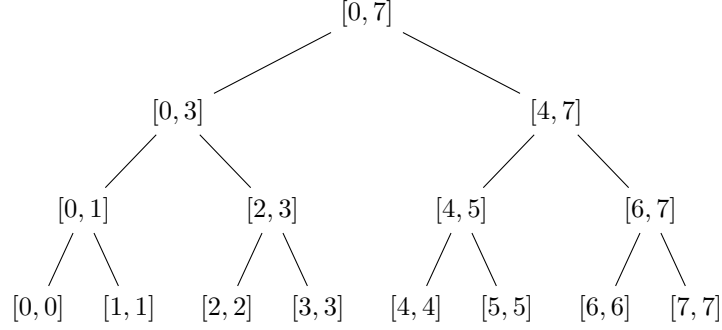
To safely store the entire tree, we typically allocate an array of size $4n$.

2.0.1 Example Structure ($n = 8$)

Suppose:

$$a = \begin{array}{|c|c|c|c|c|c|c|c|} \hline a_0 & a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 \\ \hline \end{array}$$

The segment tree intervals are:



Each node stores the minimum over its corresponding interval.

2.1 Range Query

To compute $\min(a[l \dots r])$, we start from the root, which represents the interval $[0, n - 1]$.

For a node representing interval $[L, R]$, there are three cases:

- **No overlap:** If $[L, R]$ is completely outside $[l, r]$, we return $+\infty$.
- **Complete overlap:** If $[L, R] \subseteq [l, r]$, we return the value stored in that node.
- **Partial overlap:** Otherwise, we recursively query both children:

$$\min(\text{left_result}, \text{right_result}).$$

At each level of the tree, we visit at most two nodes, and since the height of the tree is $O(\log n)$, the total time complexity is $O(\log n)$.

2.2 Point Update

To perform **set pos x**, we again start from the root, representing the interval $[0, n - 1]$.

For a node representing $[L, R]$:

- If $L = R = \text{pos}$, we update the stored value to x .
- Otherwise:

- Compute $mid = \lfloor (L + R)/2 \rfloor$.
- If $pos \leq mid$, recursively update the left child.
- Otherwise, recursively update the right child.
- After updating the child, recompute the current node as:

$$t[i] = \min(t[2i], t[2i + 1]).$$

While segment trees may seem strictly better than bucket decomposition, keep in mind that in some scenarios bucket decomposition can achieve $O(1)$ time per update (depending on the operation being supported).

In cases where the number of updates is much larger than the number of queries, this approach may outperform segment trees in practice due to its smaller constant factors and simpler structure.

3 Sparse Tables

Segment trees support both updates and range queries in $O(\log n)$ time. However, if the array is *static* (no updates), we can do even better.

Sparse tables allow us to answer range minimum queries in $O(1)$ time after $O(n \log n)$ preprocessing.

We consider the problem:

- $\min l r$ — return $\min(a[l \dots r])$.

We precompute answers for intervals whose lengths are powers of two:

$$m_{j,i} = \min(a[i \dots i + 2^j - 1])$$

That is, $m_{j,i}$ stores the minimum of the subarray starting at index i with length 2^j .

We can build the first row of the table, by simply setting $m_{0,i} = a_i$. For the rest of the table, we use the following recurrence:

$$m_{j,i} = \min(m_{j-1,i}, m_{j-1,i+2^{j-1}}).$$

So, that makes the time complexity for building the structure $O(n \log n)$.

To compute $\min(a[l \dots r])$:

- Let $k = \lfloor \log_2(r - l + 1) \rfloor$.
- We cover the interval using two blocks of length 2^k :

$$[l, l + 2^k - 1] \quad \text{and} \quad [r - 2^k + 1, r].$$

- The answer is:

$$\min(m[k][l], m[k][r - 2^k + 1]).$$

Because $\min$ is an idempotent operation, the two intervals may overlap without affecting correctness.

The query runs in $O(1)$ time.

4 Practice Problems

4.1 Sparse Table

4.1.1 Problem 1 – Range Minimum Query

You are given an array a of n integers. Answer q queries of the form (l, r) : compute $\min(a[l \dots r])$. Aim for an $O(n \log n + q)$ time complexity algorithm.

4.1.2 Problem 2 – Plantație

You are given an $n \times m$ matrix. Answer q queries of the form (i, j, k) : compute

$$\max(a[i \dots i + k - 1][j \dots j + k - 1]).$$

Aim for an $O(nm \log(nm) + q)$ time complexity algorithm.

4.1.3 Problem 3 – Strămoși

You are given a rooted tree with n nodes. Answer q queries of the form (u, k) : who is the k -th ancestor of node u ? Aim for an $O((n + q) \log n)$ time complexity solution.

4.1.4 Problem 4 – Bonus

You are given an array a of n integers and a non-idempotent function f . Answer q queries of the form (l, r) : compute $f(a[l \dots r])$. Aim for an $O(n \log n + q)$ time complexity algorithm.

4.2 Segment Trees

4.2.1 Problem 1 – Segment Trees

Implement a segment tree that supports point updates and range maximum queries. Aim for an $O((n + q) \log n)$ time complexity algorithm.

4.2.2 Problem 2 – Inv

You are given an array a of n integers. Count the number of inversions in a . An inversion is a pair (i, j) with $i < j$ such that $a_i > a_j$. Aim for an $O(n \log n)$ time complexity algorithm.

4.2.3 Problem 3 – Maxq

You are given an array a of n integers. Support q queries of the following two types:

- **set pos x** — set $a_{pos} = x$.

- **query** $l\ r$ — find the maximum sum of a contiguous subarray that is completely contained in $[l, r]$.

Aim for an $O((n + q) \log n)$ time complexity algorithm.

4.2.4 Problem 4 – Mit

You are given an array a of n integers. Support q operations of the following types:

- **add** $l\ r\ x$ — set $a_i = a_i + x$ for all $l \leq i \leq r$.
- **query** $l\ r$ — return $\max(a[l \dots r])$.

Aim for an $O((n + q) \log n)$ time complexity algorithm.

4.2.5 Problem 5 – Infinity War - bonus

Long statement, read it on Infoarena. :)